

UNIT 4

INTERMEDIATE SQL

1.Foreign Key

- A Foreign Key is a column that connects two related tables.
- It refers to the Primary Key of another table.

Example:

Users

id	name
1	John
2	May

Orders

id	user_id
101	1
102	2

Here, Orders.user_id is the Foreign Key and refers to Users.id.

2.Foreign Key Constraint

It automatically checks the relationship between two tables.

```
ALTER TABLE Orders
ADD CONSTRAINT orders_user_id_key
FOREIGN KEY (user_id)
REFERENCES Users(id);
```

Example:

If user_id = 1 is entered in Orders, id = 1 must exist in the Users table.

3.Protection from DELETE

A Foreign Key can prevent deleting a parent record when related records exist.

Example:

If John (id = 1) has an order, deleting John from Users may be prevented because his order still exists.

4.Protection from UPDATE

Foreign Key also protects related data during updates.

Example:

If Users.id = 1 is changed to 6, but Orders still contains user_id = 1, the database can prevent the update.

5.Referential Actions

These control what happens to related data during UPDATE or DELETE.

- NO ACTION → Prevents the operation.
- RESTRICT → Restricts the operation.
- CASCADE → Automatically changes related records.
- SET NULL → Changes the related value to NULL.

Example:

If User ID 1 changes to 6 and CASCADE is used:

Users.id: 1 → 6

Orders.user_id: 1 → 6

6.Transactions

- A Transaction is a group of SQL statements treated as one unit.
- It helps maintain data consistency.

Example:

In a bank transfer:

Account A → Money deducted

Account B → Money added

Both operations should happen together.

START TRANSACTION

Starts a transaction

```
START TRANSACTION;
```

Example:

```
START TRANSACTION;
```

```
UPDATE Products
```

```
SET price = 500
```

```
WHERE id = 5;
```

COMMIT

Permanently saves the transaction changes.

Example:

If the price update is correct, use COMMIT to save it permanently.

ROLLBACK

Cancels the changes made during the transaction.

Example:

If an error occurs while updating product prices, ROLLBACK returns them to their previous values.

7. ACID Properties

ACID has four properties: Atomicity, Consistency, Isolation and Durability.

Atomicity: The Smallest Unit

- All operations succeed or all fail.
- Example: Bank transfer should complete both debit and credit, or neither.

Consistency: Always Make Data Consistent

- Keeps the database in a correct state.
- Example: If a transaction fails, ROLLBACK restores the previous correct data.

Isolation: Transactions Do Not Affect Each Other

- Transactions should not improperly affect each other.
- Example: Two people withdrawing from the same account at the same time are handled separately.

Durability: Always Exist

- Committed data remains permanently stored.
- Example: After COMMIT, even if there is a power failure, the committed data should remain.

Social Media Analytics – Complete SQL Script

1.Database Design Overview

Entities

- Users
- Platforms (Instagram, X, LinkedIn, etc.)
- Posts
- Reactions (Likes, Comments, Shares)
- Followers

2. Table Creation Script (DDL)

Users Table

```
CREATE TABLE users (  
  
    user_id    INT PRIMARY KEY,  username  VARCHAR(50) NOT NULL,  
  
    join_date  DATE,   country   VARCHAR(50) );
```

Platforms Table

```
CREATE TABLE platforms ( platform_id INT PRIMARY KEY, platform_name  
VARCHAR(30) UNIQUE NOT NULL );
```

Posts Table

```
CREATE TABLE posts ( post_id  INT PRIMARY KEY,  user_id  INT,  platform_id INT,  
post_date  DATE,  content_type VARCHAR(20), -- image, video, reel, text  CONSTRAINT  
fk_post_user FOREIGN KEY (user_id) REFERENCES users(user_id),  CONSTRAINT  
fk_post_platform FOREIGN KEY (platform_id) REFERENCES platforms(platform_id) );
```

Reactions Table

```
CREATE TABLE reactions ( reaction_id INT PRIMARY KEY,  post_id  INT,  
reaction_type VARCHAR(20), -- like, comment, share  reaction_date DATE,  
CONSTRAINT fk_reaction_post FOREIGN KEY (post_id) REFERENCES posts(post_id) );
```

Followers Table

```
CREATE TABLE followers ( follower_id INT, following_id INT, follow_date DATE,  
PRIMARY KEY (follower_id, following_id), CONSTRAINT fk_follower FOREIGN KEY  
(follower_id) REFERENCES users(user_id), CONSTRAINT fk_following FOREIGN KEY  
(following_id) REFERENCES users(user_id) );
```

3.Sample Data Insertion (DML)

Users

```
INSERT INTO users VALUES (1, 'arjun_ai', DATE '2024-01-10', 'India');
```

```
INSERT INTO users VALUES (2, 'meera_data', DATE '2024-02-15', 'India');
```

```
INSERT INTO users VALUES (3, 'john_ml', DATE '2023-11-05', 'USA');
```

```
INSERT INTO users VALUES (4, 'sara_cloud', DATE '2024-03-01', 'UK');
```

Platforms

```
INSERT INTO platforms VALUES (1, 'Instagram');
```

```
INSERT INTO platforms VALUES (2, 'X');
```

```
INSERT INTO platforms VALUES (3, 'LinkedIn');
```

Posts

```
INSERT INTO posts VALUES (101, 1, 1, DATE '2024-06-01', 'reel');
```

```
INSERT INTO posts VALUES (102, 2, 1, DATE '2024-06-02', 'image');
```

```
INSERT INTO posts VALUES (103, 3, 2, DATE '2024-06-03', 'text');
```

```
INSERT INTO posts VALUES (104, 1, 3, DATE '2024-06-04', 'article');
```

Reactions

```
INSERT INTO reactions VALUES (1001, 101, 'like', DATE '2024-06-01');
```

```
INSERT INTO reactions VALUES (1002, 101, 'comment', DATE '2024-06-01');
```

```
INSERT INTO reactions VALUES (1003, 101, 'share', DATE '2024-06-02');
```

```
INSERT INTO reactions VALUES (1004, 102, 'like', DATE '2024-06-02');
```

```
INSERT INTO reactions VALUES (1005, 103, 'like', DATE '2024-06-03');
```

```
INSERT INTO reactions VALUES (1006, 104, 'comment', DATE '2024-06-04');
```

Followers

```
INSERT INTO followers VALUES (2, 1, DATE '2024-05-01');
```

```
INSERT INTO followers VALUES (3, 1, DATE '2024-05-02');
```

```
INSERT INTO followers VALUES (4, 2, DATE '2024-05-03');
```

4. Analytics Queries (Core Learning Part)

1. Total Engagement per Post

```
SELECT p.post_id, COUNT(r.reaction_id) AS total_engagement FROM posts p LEFT JOIN  
reactions r ON p.post_id = r.post_id GROUP BY p.post_id;
```

	post_id	total_engagement
▶	101	3
	102	1
	103	1
	104	1

2. Engagement Breakdown (Like / Comment / Share)

```
SELECT p.post_id, r.reaction_type, COUNT(*) AS reaction_count FROM posts p JOIN  
reactions r ON p.post_id = r.post_id GROUP BY p.post_id, r.reaction_type;
```

	post_id	reaction_type	reaction_count
▶	101	like	1
	101	comment	1
	101	share	1
	104	comment	1
	102	like	1
	103	like	1

3. Top 5 Most Engaging Posts

```
SELECT * FROM ( SELECT p.post_id, COUNT(r.reaction_id) AS engagement, RANK() OVER
(ORDER BY COUNT(r.reaction_id) DESC) AS rnk FROM posts p LEFT JOIN reactions r ON
p.post_id = r.post_id GROUP BY p.post_id ) WHERE rnk <= 5;
```

	post_id	engagement	rnk
▶	101	3	1
	104	1	2
	102	1	2
	103	1	2

4. Most Engaging Users

```
SELECT u.username, COUNT(r.reaction_id) AS total_engagement FROM users u JOIN
posts p ON u.user_id = p.user_id LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY
u.username ORDER BY total_engagement DESC;
```

	username	total_engagement
▶	arjun_ai	4
	meera_data	1
	john_ml	1

5. Users Who Never Posted

SELECT u.username FROM users u LEFT JOIN posts p ON u.user_id = p.user_id WHERE p.post_id IS NULL;

	username
▶	sara_cloud

6. Average Reactions per Post

SELECT AVG(reaction_count) AS avg_reactions FROM (SELECT post_id, COUNT() AS reaction_count FROM reactions GROUP BY post_id);

	avg_reactions
▶	1.5000

7. Platform-wise Engagement

SELECT pl.platform_name, COUNT(r.reaction_id) AS total_engagement FROM platforms pl JOIN posts p ON pl.platform_id = p.platform_id LEFT JOIN reactions r ON p.post_id = r.post_id GROUP BY pl.platform_name;

	platform_name	total_engagement
▶	Instagram	4
	LinkedIn	1
	X	1

8. Peak Posting Day

SELECT post_date, COUNT() AS post_count FROM posts GROUP BY post_date ORDER BY post_count DESC;

	post_date	post_count
▶	2024-06-01	1
	2024-06-02	1
	2024-06-03	1
	2024-06-04	1

9. Users with More Than 1,000 Engagements (Scalable Query)

SELECT u.username FROM users u JOIN posts p ON u.user_id = p.user_id JOIN reactions r ON p.post_id = r.post_id GROUP BY u.username HAVING COUNT(r.reaction_id) > 1000;

	username
--	----------

10. Followers Count per User

SELECT u.username, COUNT(f.follower_id) AS follower_count FROM users u LEFT JOIN followers f ON u.user_id = f.following_id GROUP BY u.username;

	username	follower_count
▶	arjun_ai	2
	meera_data	1
	john_ml	0
	sara_cloud	0